

MapReduce with MongoDB

Insert the following details into MongoDB:

Order ID	Customer	Order Date	Order Total	Item	Quantity	Unit Price	Status
1	Ant O. Knee	3/1/2020	25	oranges	5	2.5	A
1	Ant O. Knee	3/1/2020	25	apples	5	2.5	A
2	Ant O. Knee	3/8/2020	70	oranges	8	2.5	A
2	Ant O. Knee	3/8/2020	70	chocolates	5	10	A
3	Busby Bee	3/8/2020	50	oranges	10	2.5	A
3	Busby Bee	3/8/2020	50	pears	10	2.5	A
4	Busby Bee	3/18/2020	25	oranges	10	2.5	A
5	Busby Bee	3/19/2020	50	chocolates	5	10	A
6	Cam Elot	3/19/2020	35	carrots	10	1	A
6	Cam Elot	3/19/2020	35	apples	10	2.5	A
7	Cam Elot	3/20/2020	25	oranges	10	2.5	A
8	Don Quis	3/20/2020	75	chocolates	5	10	A
8	Don Quis	3/20/2020	75	apples	10	2.5	A
9	Don Quis	3/20/2020	55	carrots	5	1	A
9	Don Quis	3/20/2020	55	apples	10	2.5	A
9	Don Quis	3/20/2020	55	oranges	10	2.5	A
10	Don Quis	3/23/2020	25	oranges	10	2.5	A

```
db.orders.insertMany([
  { _id: 1, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-01"), price:
25, items: [ { sku: "oranges", qty: 5, price: 2.5 }, { sku: "apples", qty: 5, price:
2.5 } ], status: "A" },
  { _id: 2, cust_id: "Ant O. Knee", ord_date: new Date("2020-03-08"), price:
70, items: [ { sku: "oranges", qty: 8, price: 2.5 }, { sku: "chocolates", qty: 5,
price: 10 } ], status: "A" },
  { _id: 3, cust_id: "Busby Bee", ord_date: new Date("2020-03-08"), price: 50,
items: [ { sku: "oranges", qty: 10, price: 2.5 }, { sku: "pears", qty: 10, price: 2.5
} ], status: "A" },
  { _id: 4, cust_id: "Busby Bee", ord_date: new Date("2020-03-18"), price: 25,
items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 5, cust_id: "Busby Bee", ord_date: new Date("2020-03-19"), price: 50,
items: [ { sku: "chocolates", qty: 5, price: 10 } ], status: "A"},
  { _id: 6, cust_id: "Cam Elot", ord_date: new Date("2020-03-19"), price: 35,
items: [ { sku: "carrots", qty: 10, price: 1.0 }, { sku: "apples", qty: 10, price: 2.5
} ], status: "A" },
  { _id: 7, cust_id: "Cam Elot", ord_date: new Date("2020-03-20"), price: 25,
items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 8, cust_id: "Don Quis", ord_date: new Date("2020-03-20"), price: 75,
items: [ { sku: "chocolates", qty: 5, price: 10 }, { sku: "apples", qty: 10, price:
2.5 } ], status: "A" },
  { _id: 9, cust_id: "Don Quis", ord_date: new Date("2020-03-20"), price: 55,
items: [ { sku: "carrots", qty: 5, price: 1.0 }, { sku: "apples", qty: 10, price: 2.5
}, { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" },
  { _id: 10, cust_id: "Don Quis", ord_date: new Date("2020-03-23"), price: 25,
items: [ { sku: "oranges", qty: 10, price: 2.5 } ], status: "A" }
])
```

Return the Total Price Per Customer

Perform the map-reduce operation on the orders collection to group by the cust_id, and calculate the sum of the price for each cust_id:

1. Define the map function to process each input document:
 - In the function, this refers to the document that the map-reduce operation is processing.
 - The function maps the price to the cust_id for each document and emits the cust_id and price.

```
var mapFunction1 = function() {  
  emit(this.cust_id, this.price);  
};
```

□ Define the corresponding reduce function with two arguments keyCustId and valuesPrices:

- The valuesPrices is an array whose elements are the price values emitted by the map function and grouped by keyCustId.
- The function reduces the valuesPrice array to the sum of its elements.

```
var reduceFunction1 = function(keyCustId, valuesPrices) {  
  return Array.sum(valuesPrices);  
};
```

Perform map-reduce on all documents in the orders collection using the mapFunction1 map function and the reduceFunction1 reduce function:

```
db.orders.mapReduce(  
  mapFunction1,  
  reduceFunction1,  
  { out: "map_reduce_example" }  
)
```

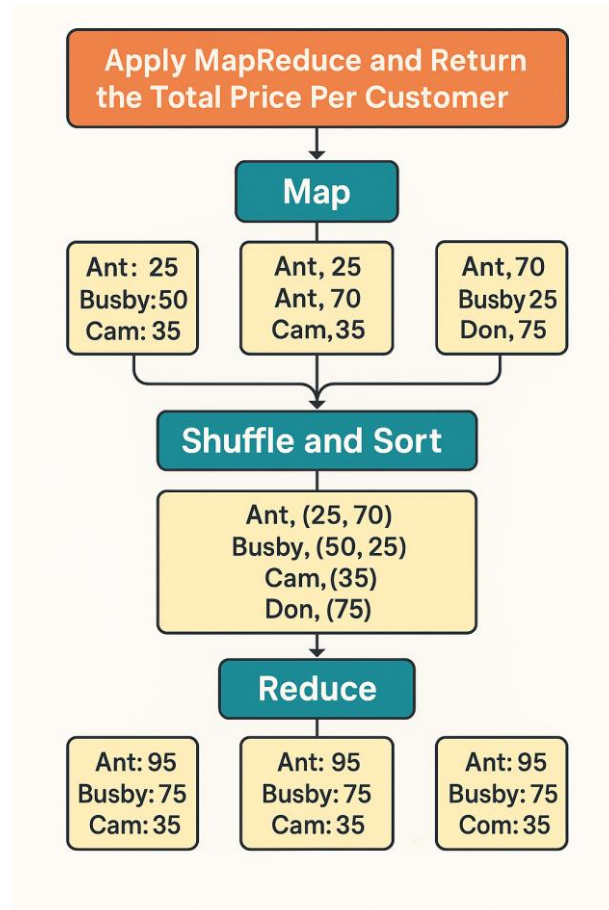
□ This operation outputs the results to a collection named map_reduce_example. If the map_reduce_example collection already exists, the operation will replace the contents with the results of this map-reduce operation.

□ Query the map_reduce_example collection to verify the results:

```
db.map_reduce_example.find().sort( { _id: 1 } )
```

4. The operation returns these documents:

```
{ "_id" : "Ant O. Knee", "value" : 95 }  
{ "_id" : "Busby Bee", "value" : 125 }  
{ "_id" : "Cam Elot", "value" : 60 }  
{ "_id" : "Don Quis", "value" : 155 }
```



Aggregation Alternative

Using the available aggregation pipeline operators, you can rewrite the map-reduce operation without defining custom functions:

```
db.orders.aggregate([
  { $group: { _id: "$cust_id", value: { $sum: "$price" } } },
  { $out: "agg_alternative_1" }
])
```

□ The [\\$group](#) stage groups by the `cust_id` and calculates the value field (See also [\\$sum](#)). The value field contains the total price for each `cust_id`.

The stage output the following documents to the next stage:

```
{ "_id" : "Don Quis", "value" : 155 }
{ "_id" : "Ant O. Knee", "value" : 95 }
{ "_id" : "Cam Elot", "value" : 60 }
{ "_id" : "Busby Bee", "value" : 125 }
```

□ Then, the [\\$out](#) writes the output to the collection `agg_alternative_1`.

Alternatively, you could use [\\$merge](#) instead of [\\$out](#).

□ Query the `agg_alternative_1` collection to verify the results:

```
db.agg_alternative_1.find().sort( { _id: 1 } )
```

3. The operation returns the following documents:

```
{ "_id" : "Ant O. Knee", "value" : 95 }
{ "_id" : "Busby Bee", "value" : 125 }
{ "_id" : "Cam Elot", "value" : 60 }
{ "_id" : "Don Quis", "value" : 155 }
```